

Does the **Order of Conditions** in a **WHERE** Clause Matter?

In PostgreSQL





```
where d.status == DeliveryStatus.Received  
      && o.StoreId == user.Id
```



```
where o.StoreId == user.Id  
      && d.status == DeliveryStatus.Received
```

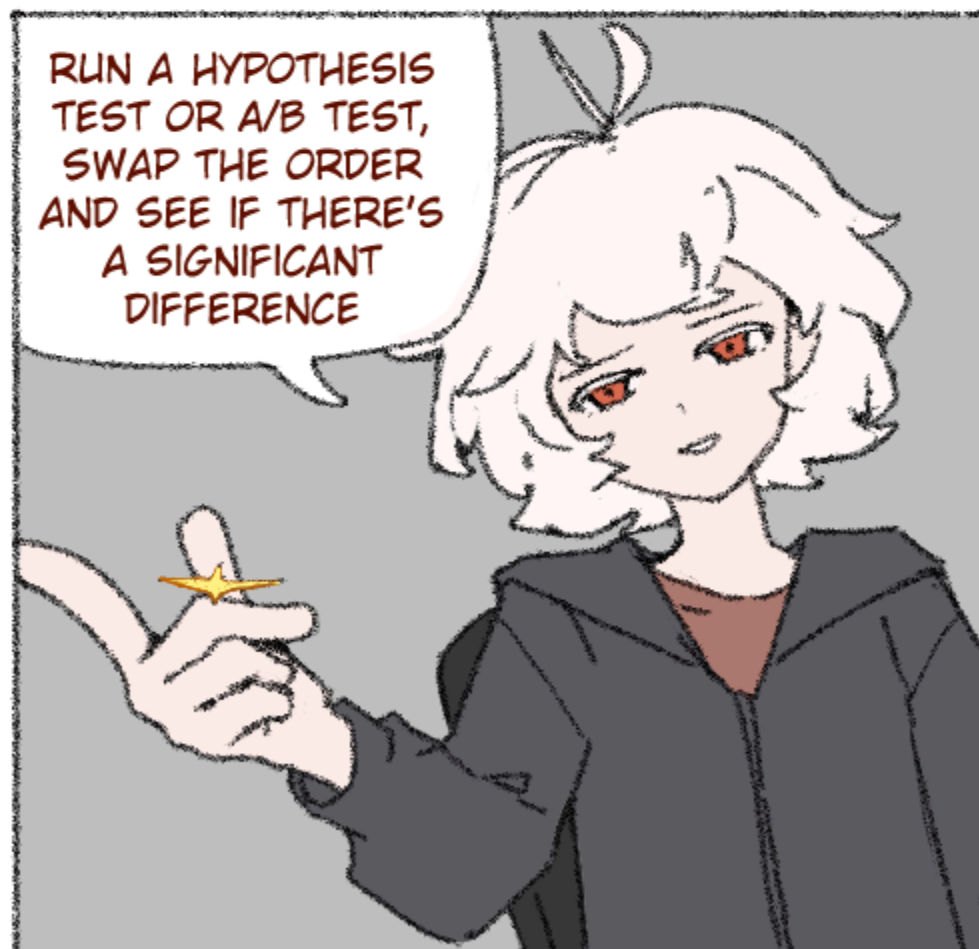
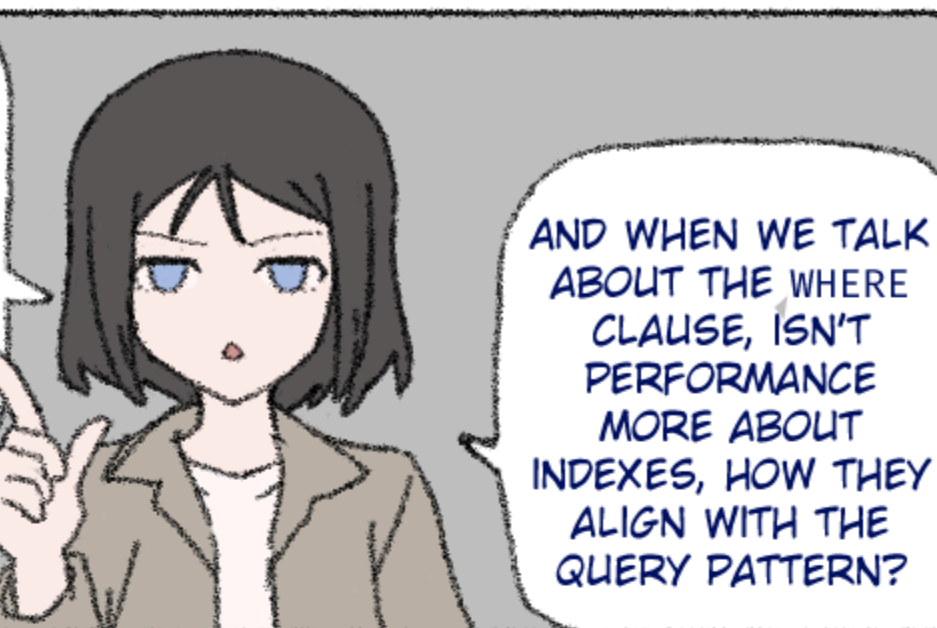
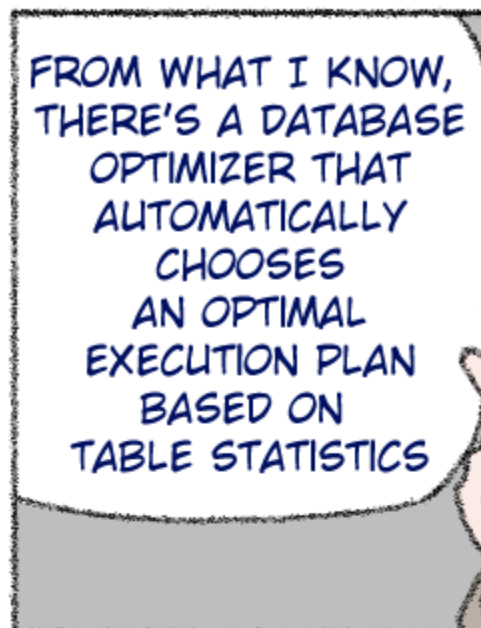
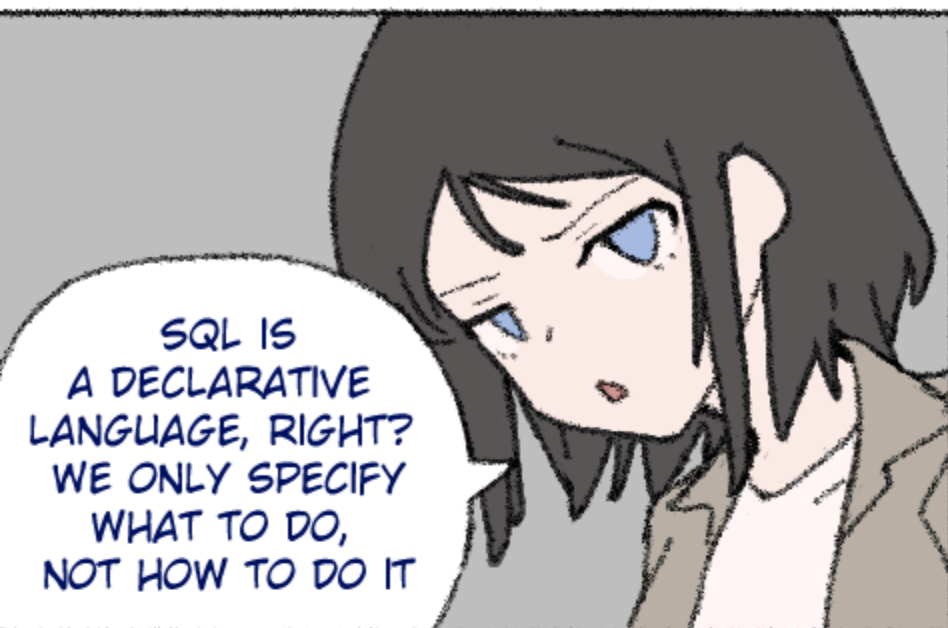
I NEED YOU TO CHANGE THE ORDER
OF THESE TWO CONDITIONS IN THE WHERE CLAUSE

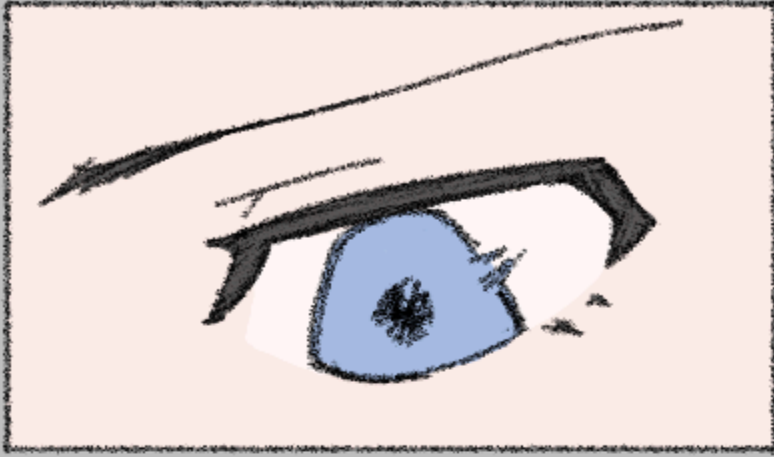


WHY? DOES
IT MATTER?

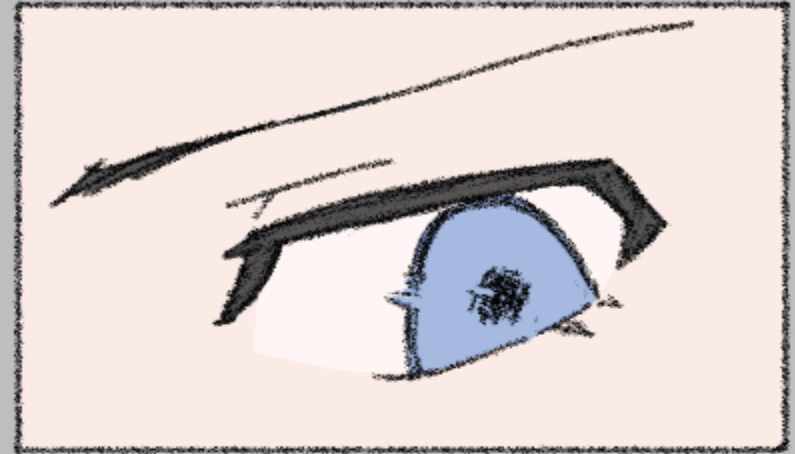
WELL, IF WE PUT THE SELECTIVE
CONDITION FIRST, IT'LL DROPS MOST OF
THE ROWS SO WE DON'T HAVE TO
EVALUATE THE SECOND CONDITION

I DON'T THINK THAT'S
HOW DATABASES WORK





SO IT'S CALLED A HYPOTHESIS TEST
BECAUSE YOU MUST STATE IT
BEFORE TESTING IT

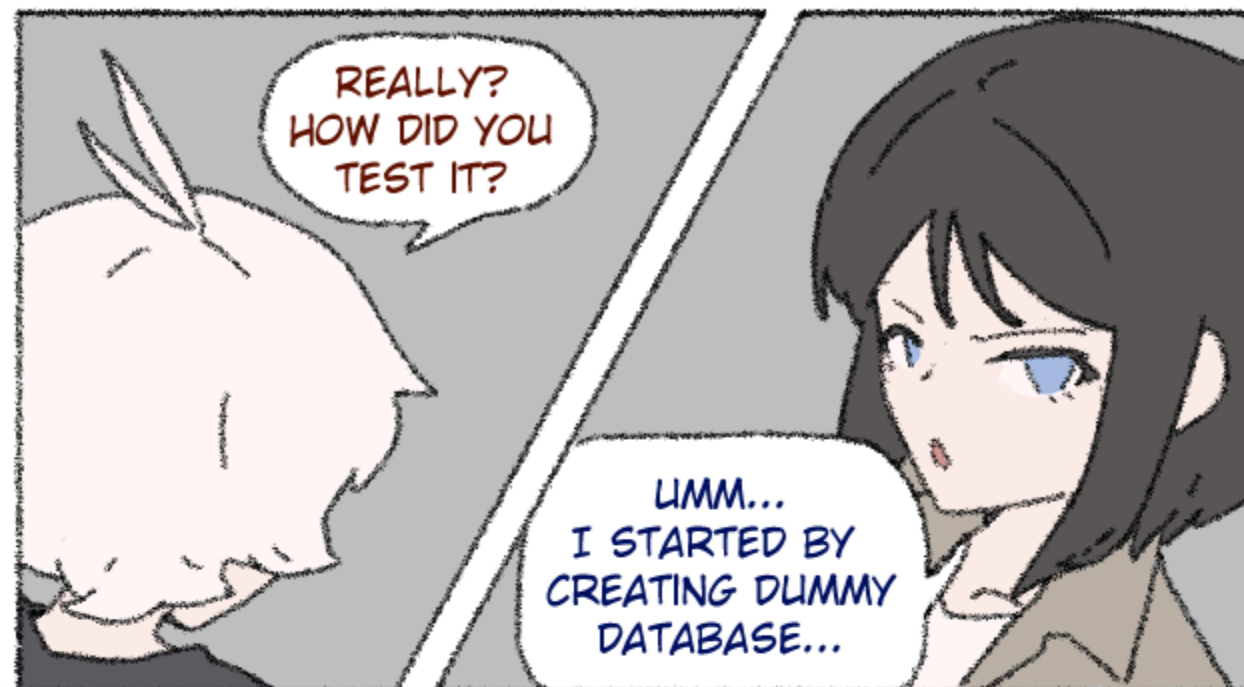
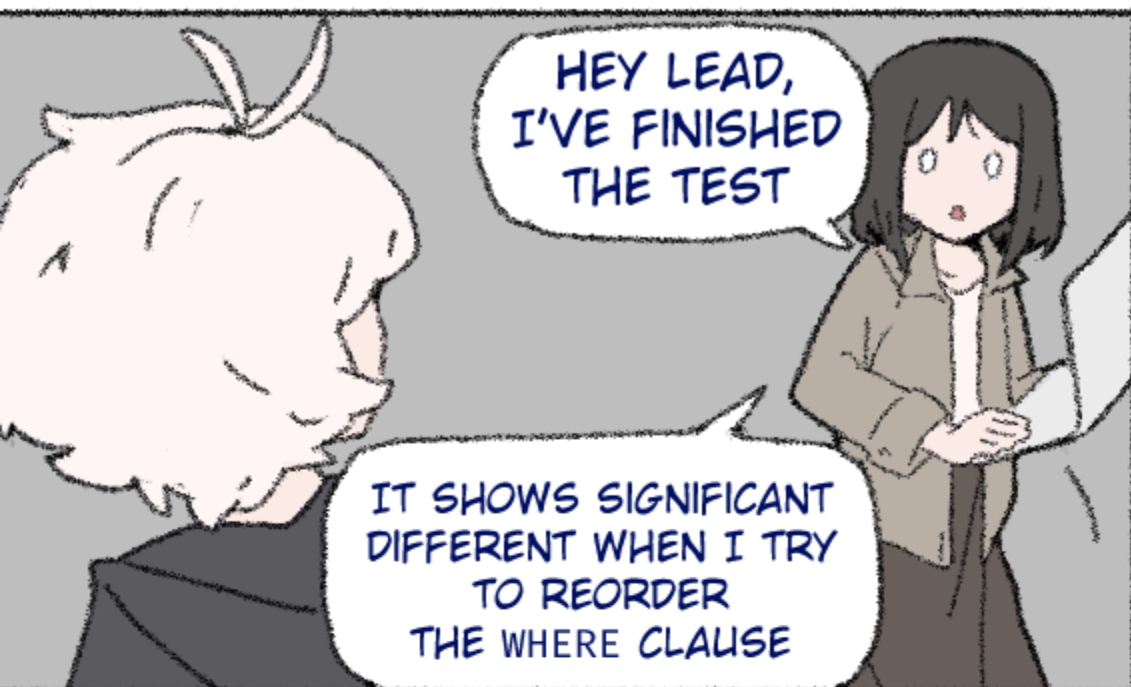


BESIDES THE HYPOTHESIS,
WE ALSO NEED TO CHOOSE A SIGNIFICANCE LEVEL,
WHICH DEFINES THE PROBABILITY OF INCORRECTLY
REJECTING THE NULL HYPOTHESIS

HOW MANY
SAMPLES
DO WE NEED...?



I SEE...
SO WE CAN USE A T-TEST
IS THERE A LIBRARY
FOR THAT?



THERE'S orders TABLE THAT I SEED WITH 2 MILLION SYNTHETIC RECORD LIKE THIS

```
INSERT INTO orders (customer_id, order_date, status)
SELECT
  (gs % 100000) + 1,
  NOW() - ((gs % 365) || ' days')::INTERVAL,
  CASE
    WHEN gs % 10 < 9 THEN 'completed'
    WHEN gs % 10 = 9 THEN 'pending'
    ELSE 'cancelled'
  END
FROM generate_series(1, 2000000) gs;
```

IT GENERATED ORDERS WHERE customer_id CYCLES FROM 1 TO 100,000, AND status IS MOSTLY 'completed' (90%) WITH A SMALLER PORTION 'pending' (10%)

THEN I TRIED QUERYING THE orders USING THOSE TWO COLUMNS AS FILTER PREDICATES

I WROTE TWO SIMPLE QUERIES THAT DIFFER ONLY IN THE ORDER OF THE COLUMNS IN THE WHERE CLAUSE

```
query_a = f"""
SELECT COUNT(*)
FROM orders
WHERE customer_id = {customer_id}
  AND status = 'completed'
"""
```

```
query_b = f"""
SELECT COUNT(*)
FROM orders
WHERE status = 'completed'
  AND customer_id = {customer_id}
"""
```

I USED PYTHON TO EXECUTE THE QUERIES, THEN TO MINIMIZE DATA TRANSFER OVERHEAD BETWEEN PYTHON AND THE DATABASE, I ONLY SELECTED COUNT(*)



I EXECUTED BOTH QUERIES 100 TIMES ACROSS 30 DIFFERENT RANDOM SEEDS, RESULTING IN 3,000 SAMPLES

0.028478
0.02878
0.028485
0.0263822
0.02713641800073674,0.03461339500063332
0.027187267001863802,0.03504762599914102
0.0340830389977782,0.03497786599837127
0.03116078200037009,0.03649355200104765
0.027402835999964736,0.03559475599831785
0.02704021500176168,0.035538772001018515
0.02758844100026181,0.03556395700070425
0.02713641800073674,0.03461339500063332
0.027187267001863802,0.03504762599914102
0.0340830389977782,0.03497786599837127
0.03116078200037009,0.03649355200104765
0.027402835999964736,0.03559475599831785
0.02704021500176168,0.035538772001018515
0.02758844100026181,0.03556395700070425

THEN I RECORDED THE EXECUTION TIME FOR EACH RUN

WAIT, WHY DO YOU NEED RANDOM SEEDS?

OH, THERE'S OS MEMORY MANAGEMENT, RIGHT? AND QUERYING THE DATABASE INVOLVES MOVING DATA FROM DISK TO MEMORY. SO THE SECOND EXECUTION MIGHT BENEFIT FROM CACHED DATA

SO I RANDOMIZED THE EXECUTION ORDER SO BOTH QUERIES HAD AN EQUAL CHANCE OF BEING FIRST OR SECOND

I'M NOT SURE HOW POSTGRESQL MANAGES ITS MEMORY THOUGH, BUT I'M JUST HOPING THIS HELPS REMOVE ANY BIAS FROM THE EXECUTION ORDER

GOOD POINT

AFTER I FINISHED COLLECTING MY SAMPLE. I USED `ttest_rel()` FUNCTION FROM SciPy LIBRARY TO PERFORM THE HYPOTESIS TEST

I CHOSE SIGNIFICANCE LEVEL OF 0.05, SO WE WOULD REJECT THE NULL HYPOTHESIS, THAT FILTER ORDER HAS NO EFFECT, IF THE P-VALUE IS ≤ 0.05 .

```
if filter_p_value < 0.05:  
    print("Reject H0 → significant difference detected")  
else:  
    print("Fail to reject H0 → no significant difference")
```

HERE'S THE RESULT

T-statistic : -193.82125833117516
P-value : 0.0
Reject H0 → significant difference detected

AS YOU CAN SEE, THE P-VALUE IS ZERO, INDICATING A STATISTICALLY SIGNIFICANT DIFFERENCE BETWEEN THE TWO QUERIES

IF WE LOOK AT THE SAMPLE DATA, WE'LL SEE THAT THE FIRST QUERY, WHERE WE EVALUATE `customer_id` FIRST IS ACTUALLY FASTER THAN THE SECOND QUERY

execution_order	time_a	time_b
B_then_A	0.026423960000101943	0.04143874100554967
B_then_A	0.026907118000963237	0.035086748997855466
A_then_B	0.028552233998198062	0.03573491900169756
B_then_A	0.026218427003186662	0.034324619999097195
B_then_A	0.02613925499463221	0.03735286400478799

I SEE... SO THE ORDER REALLY MATTERS, HUH?

THE RESULTS SAY SO, BUT I'M STILL NOT FULLY CONVINCED

I STILL BELIEVE THE PLANNER SHOULD DETERMINE THE OPTIMAL EXECUTION PLAN, INCLUDING THE ORDER OF THE FILTERS

WITH THE SIGNIFICANCE LEVEL I USED, THERE'S STILL A 5% CHANCE THE RESULT IS JUST BECAUSE OF LUCK

I ALSO CHECKED THE QUERY PLAN AND HERE'S WHAT I FOUND

```
dummy_ecommerce=# EXPLAIN
SELECT COUNT(*)
FROM orders
WHERE customer_id = 5173 AND status = 'completed';
QUERY PLAN

Finalize Aggregate (cost=30029.24..30029.24 rows=1 width=8)
  → Gather (cost=30029.02..30029.23 rows=2 width=8)
    Workers Planned: 2
    → Partial Aggregate (cost=29029.02..29029.03 rows=1 width=8)
      → Parallel Seq Scan on orders (cost=0.00..29029.00 rows=8 width=0)
        Filter: ((customer_id = 5173) AND (status = 'completed'::text))
```

(6 rows)

```
dummy_ecommerce=# EXPLAIN
SELECT COUNT(*)
FROM orders
WHERE status = 'completed' AND customer_id = 5173;
QUERY PLAN

Finalize Aggregate (cost=30029.24..30029.24 rows=1 width=8)
  → Gather (cost=30029.02..30029.23 rows=2 width=8)
    Workers Planned: 2
    → Partial Aggregate (cost=29029.02..29029.03 rows=1 width=8)
      → Parallel Seq Scan on orders (cost=0.00..29029.00 rows=8 width=0)
        Filter: ((status = 'completed'::text) AND (customer_id = 5173))
```

(6 rows)

dummy_ecommerce=#

IN THE FILTER SECTION, THE PREDICATE ORDER REMAINS THE SAME AS IN THE WHERE CLAUSE

SO I THINK THE FIRST QUERY IS FASTER BECAUSE ITS FIRST CONDITION IS MORE SELECTIVE AND DUE TO SHORT-CIRCUIT EVALUATION, THE DATABASE MAY AVOID EVALUATING THE SECOND CONDITION

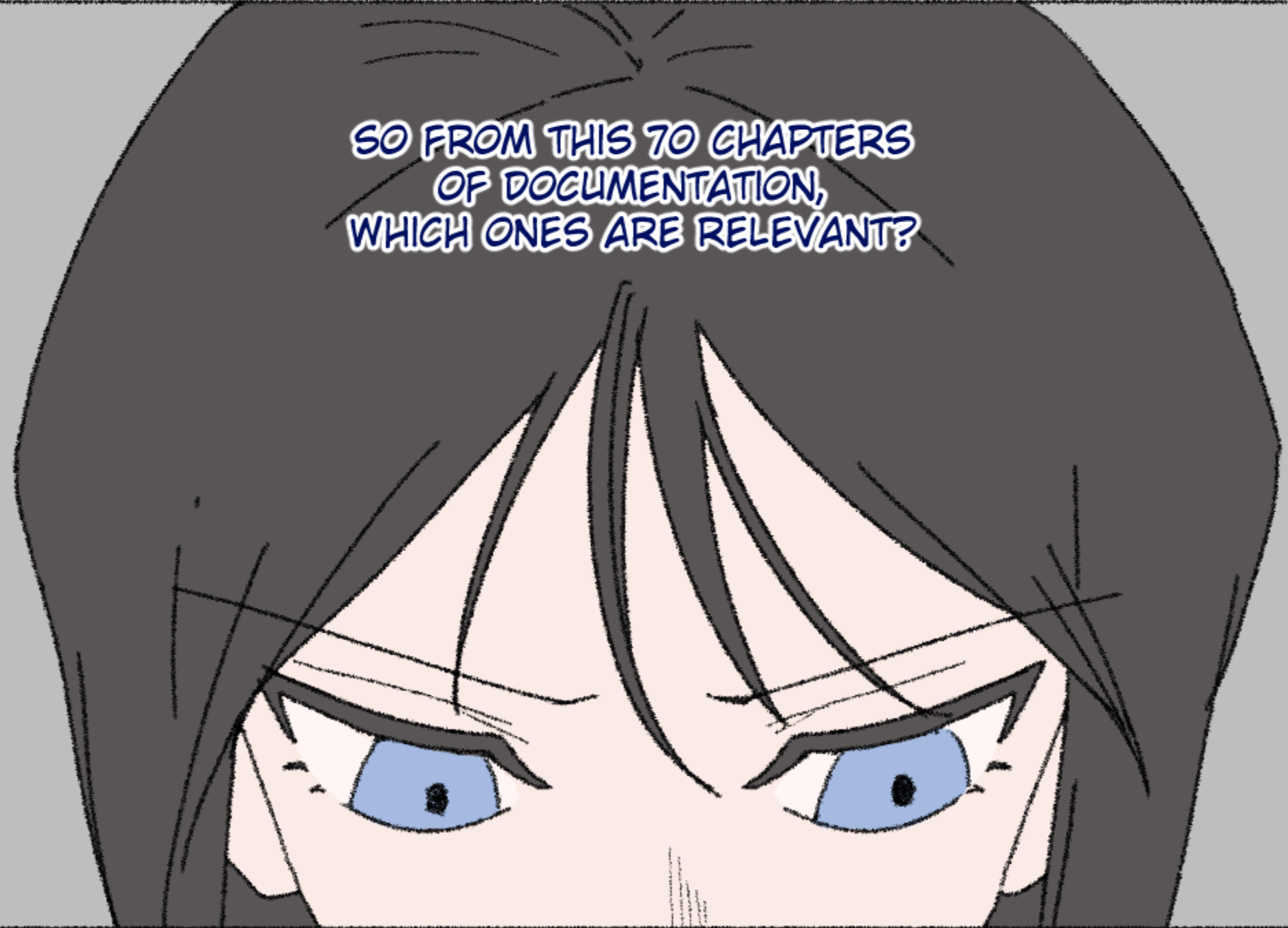
SO NOW THE QUESTION IS: WHY WAS NO REORDERING PERFORMED?



I'LL JUST START READING
THE DOCUMENTATION TO FIND OUT

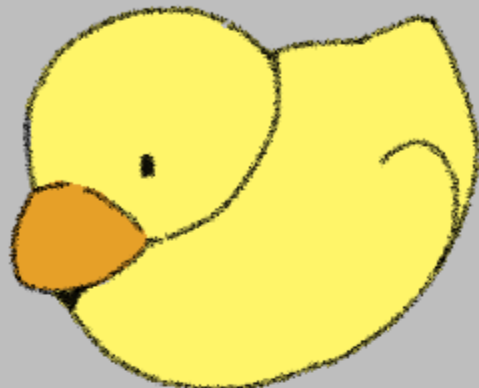
I SEE... GO AHEAD,
BUT DON'T FORGET YOU STILL
HAVE OTHER TASKS THIS WEEK

ALRIGHT

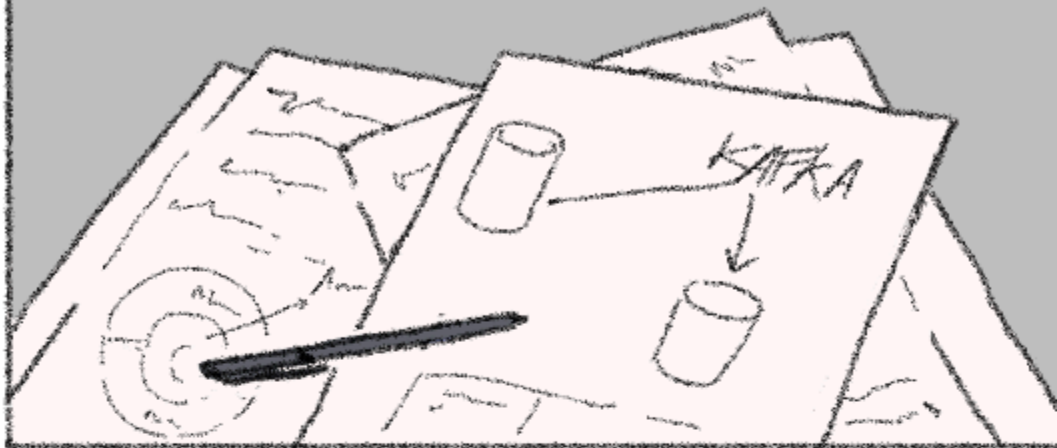


SO FROM THIS 70 CHAPTERS
OF DOCUMENTATION,
WHICH ONES ARE RELEVANT?

"THE ORDER OF EVALUATION OF SUBEXPRESSIONS IS NOT DEFINED. IN PARTICULAR, THE INPUTS OF AN OPERATOR OR FUNCTION ARE NOT NECESSARILY EVALUATED LEFT-TO-RIGHT OR IN ANY OTHER FIXED ORDER"



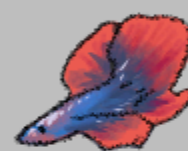
SEE? THE ORDER IN THE WHERE CLAUSE DOESN'T MATTER, RIGHT? THEN WHY DOESN'T THE PLANNER JUST REORDER IT INTO THE OPTIMAL ONE FOR THESE QUERIES?



"FURTHERMORE, IF THE RESULT OF AN EXPRESSION CAN BE DETERMINED BY EVALUATING ONLY SOME PARTS OF IT, THEN OTHER SUBEXPRESSIONS MIGHT NOT BE EVALUATED AT ALL"



THIS MIGHT EXPLAINS WHY THE FIRST QUERY WAS FASTER



OH, THIS PART IS ABOUT TUNING PARAMETERS TO USE PARALLEL COMPUTATION. I THINK OUR DB CPU ISN'T FULLY UTILIZED YET MAYBE WE CAN TRY IT...?

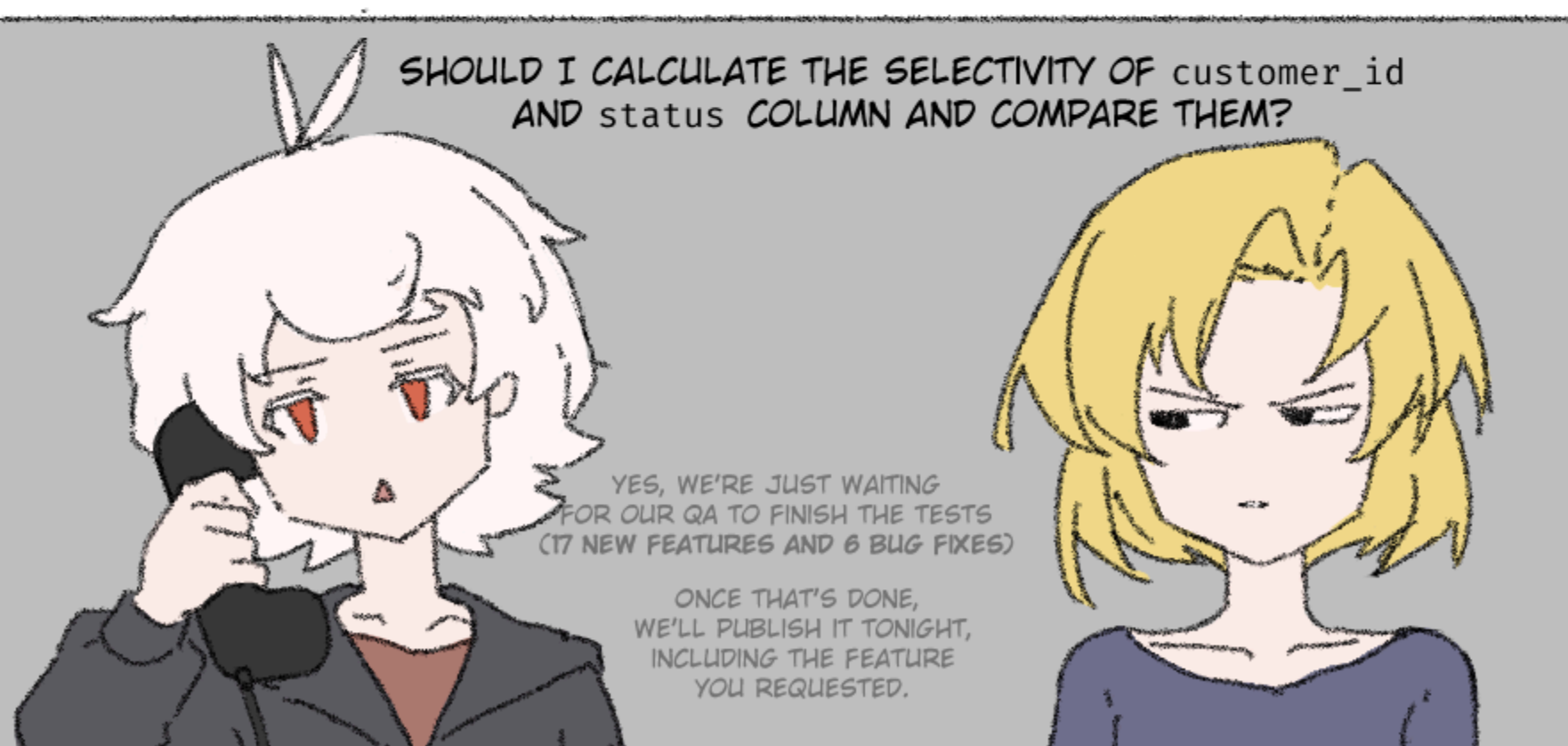
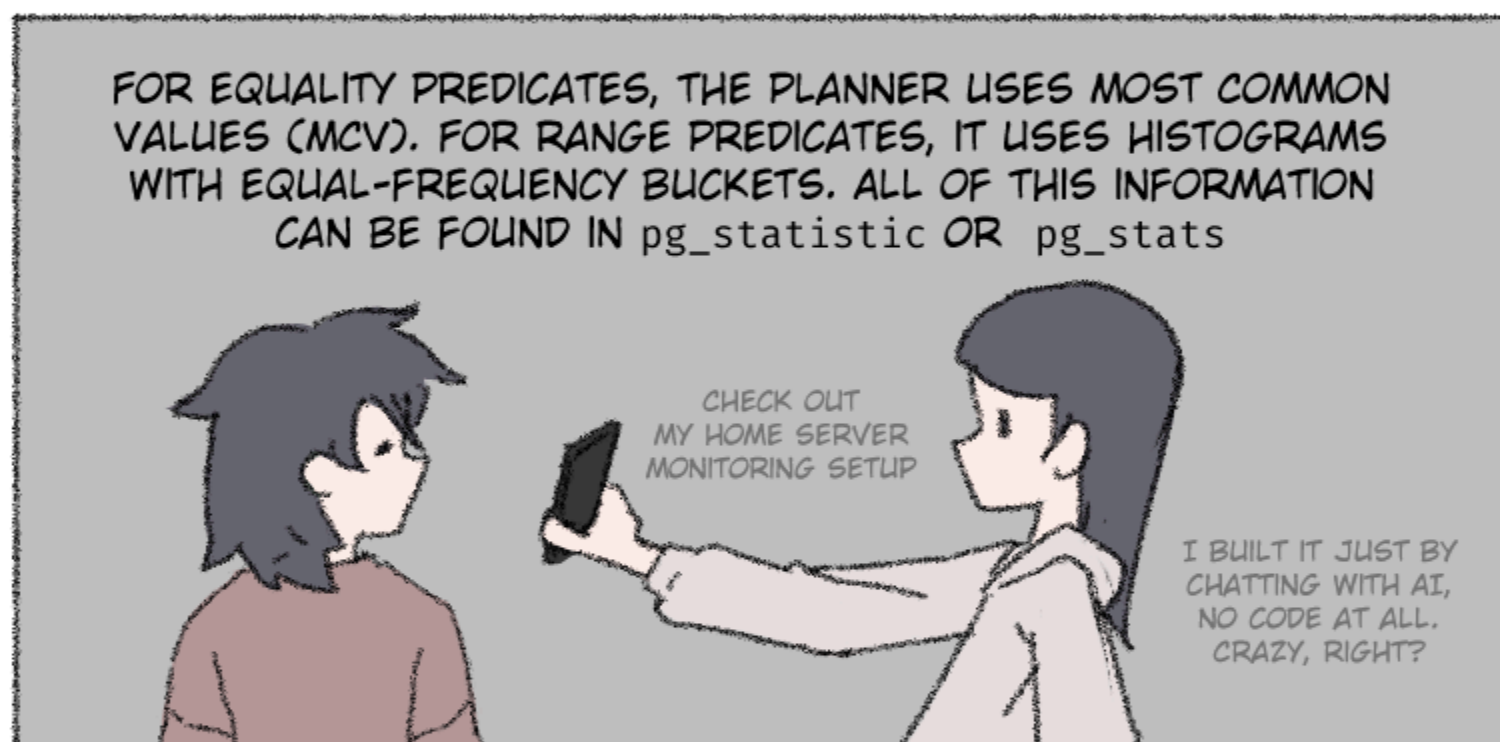
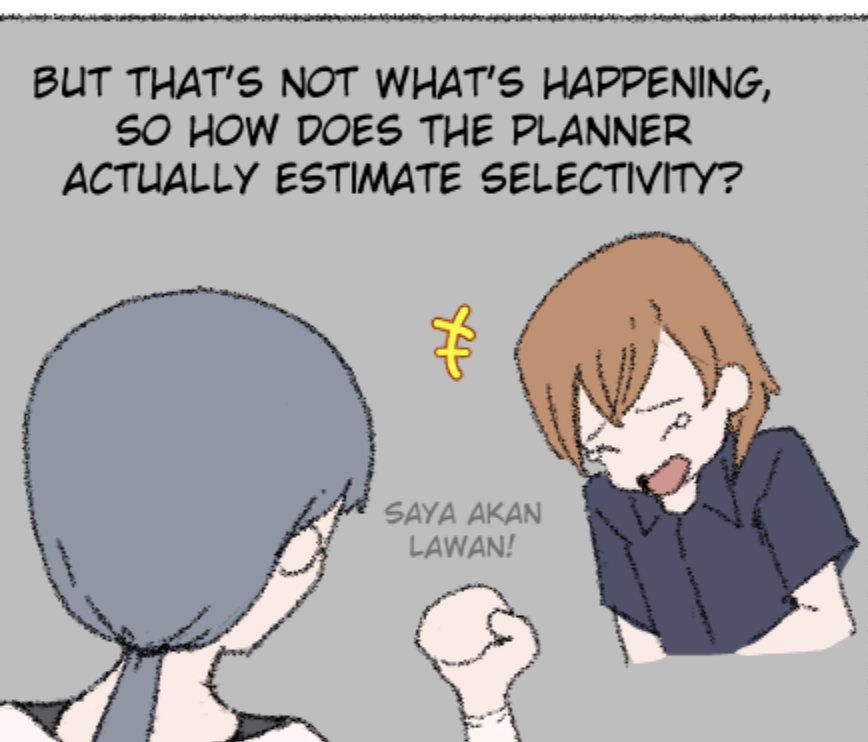
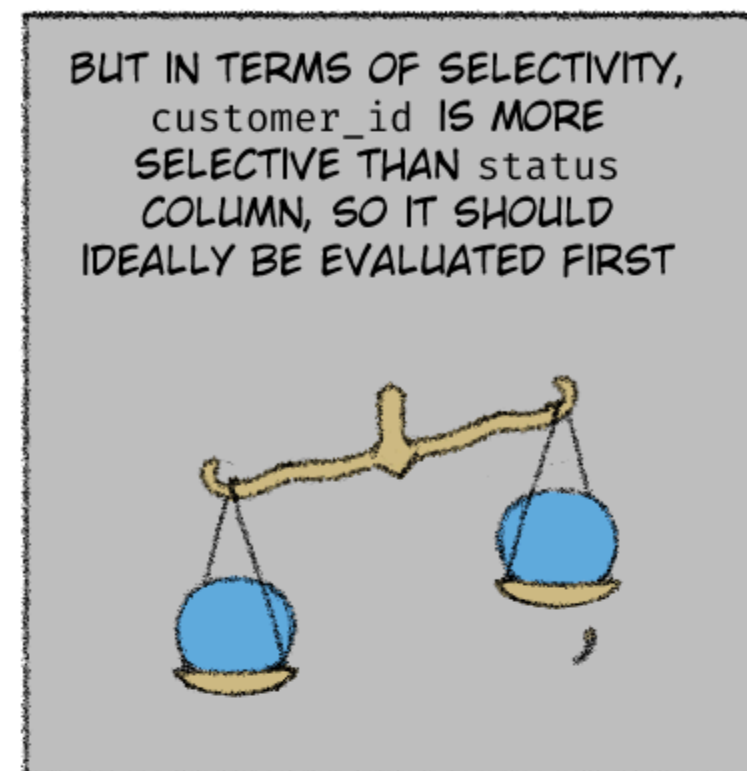
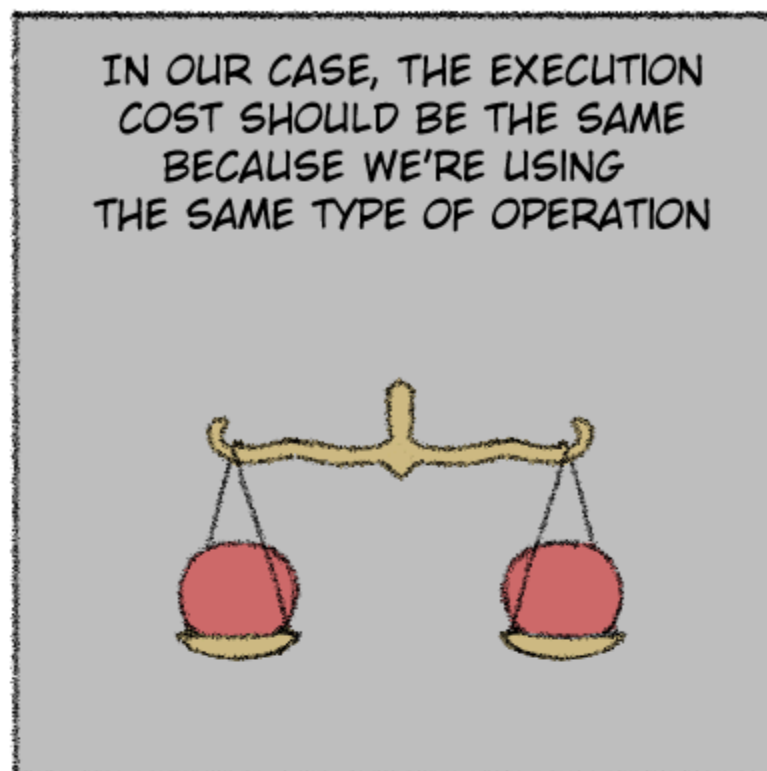
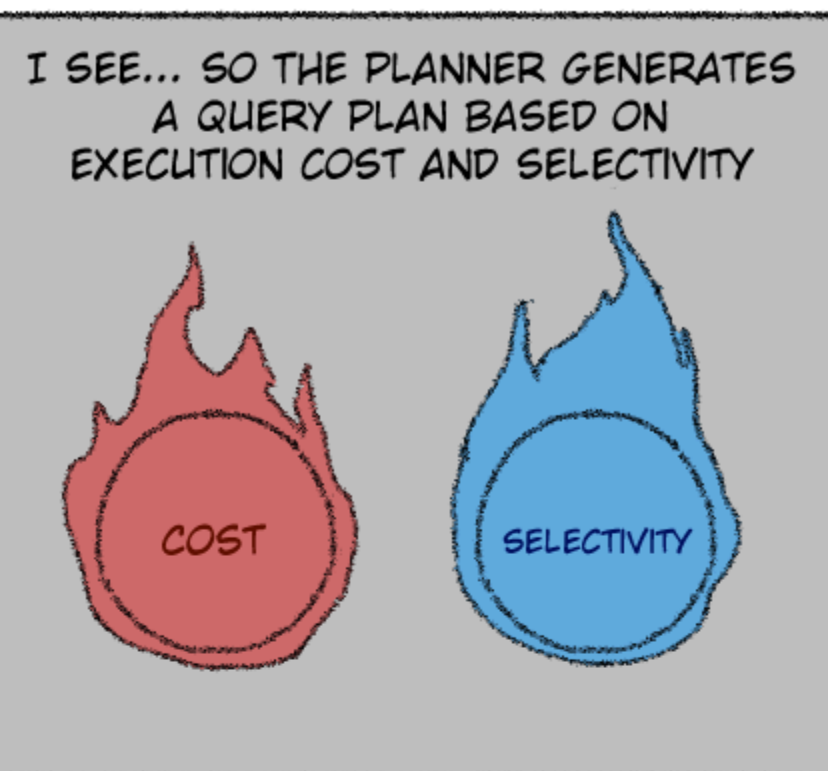
I'M ABOUT TO ORDER SOME SNACKS WANT ANYTHING?



BUT LET'S FOCUS ON THIS FIRST

REALLY? IT COST ME \$7 JUST FOR THIS UNIT TESTS I DON'T EVEN UNDERSTAND?!





PostgreSQL 18.4 Documentation:

- [14.1. Using EXPLAIN](#)
- [69.1. Row Estimation Examples](#)
- [14.2.1. Single-Column Statistics](#)

I DON'T REALLY CARE WHICH PREDICATE IS MORE SELECTIVE

THE PLANNER CAN DECIDE WHICH ONE TO EVALUATE FIRST

THE POINT IS, SOMEHOW THESE TWO QUERIES DIDN'T PRODUCE A CONSISTENT FILTER ORDER

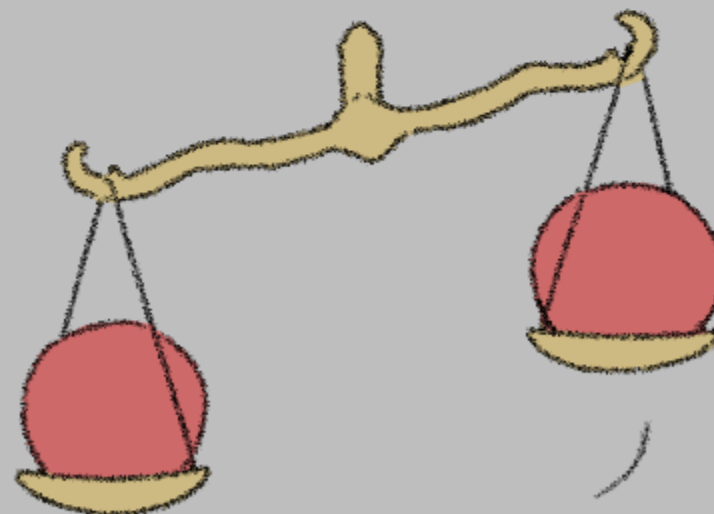
DOES THAT MEAN SELECTIVITY ISN'T USED AT ALL HERE?

THAT'S IT

WELL IF THE PLANNER ONLY ARRANGES THE ORDER BASED ON EXECUTION COST, THEN IT MAKES SENSE THAT NO REORDERING HAPPENS AS THEY HAVE IDENTICAL COST

OKAY, LET'S MAKE THE COSTS DIFFERENT. WE'LL INTRODUCE A FUNCTION INTO THE `customer_id` PREDICATE, BY FILTERING BASED ON THE CURRENT DAY USING THE `NOW()` FUNCTION

```
WHERE customer_id = EXTRACT(DAY FROM NOW())
```



THAT WILL MAKE THE `customer_id` PREDICATE MORE EXPENSIVE TO EXECUTE, SO WE COULD EXPECT THE PLANNER TO PUSH IT LATER IN THE EVALUATION ORDER

```
dummy_ecommerce=# EXPLAIN
SELECT COUNT(*)
FROM orders
WHERE customer_id = EXTRACT(DAY FROM NOW()) AND status = 'completed'
QUERY PLAN
```

```
Finalize Aggregate (cost=36288.57..36288.58 rows=1 width=8)
  → Gather (cost=36288.36..36288.57 rows=2 width=8)
    Workers Planned: 2
    → Partial Aggregate (cost=35288.36..35288.37 rows=1 width=8)
      → Parallel Seq Scan on orders (cost=0.00..35279.00 rows=3744 width=0)
        Filter: ((status = 'completed'::text) AND ((customer_id)::numeric = EXTRACT(day FROM now())))
(6 rows)
```

```
dummy_ecommerce=# EXPLAIN
SELECT COUNT(*)
FROM orders
WHERE status = 'completed' AND customer_id = EXTRACT(DAY FROM NOW());
QUERY PLAN
```

```
Finalize Aggregate (cost=36288.57..36288.58 rows=1 width=8)
  → Gather (cost=36288.36..36288.57 rows=2 width=8)
    Workers Planned: 2
    → Partial Aggregate (cost=35288.36..35288.37 rows=1 width=8)
      → Parallel Seq Scan on orders (cost=0.00..35279.00 rows=3744 width=0)
        Filter: ((status = 'completed'::text) AND ((customer_id)::numeric = EXTRACT(day FROM now())))
(6 rows)
```

```
dummy_ecommerce=#
```

NICE... THE RESULT MATCHES OUR EXPECTATION

WE GOT A CONSISTENT RESULT

DID YOU FIND IT?

THE ANSWER

NOT YET

I HAVE A SPECULATION, BUT I AM NOT SURE HOW TO PROVE IT

YOU KNOW POSTGRESQL IS OPEN-SOURCE, RIGHT? MAYBE CHECKING THE SOURCE CODE DIRECTLY MIGHT HELP?

YOU CAN USE AI TO HELP POINT YOU TO THE RELEVANT PARTS

HEY, THAT'S A GOOD IDEA!

SO WE ARE DIRECTED TO THIS `order_qual_clauses()` FUNCTION

LET'S SEE... "QUAL" REFERS TO EXPRESSIONS IN THE WHERE CLAUSE

AH! FOUND IT!

REALLY?

LOOK HERE!

...als that are Leakproof. When security_level is on, we prefer to order clauses by estimated cost first.

* Ideally the order should be driven by a combination of execution cost and selectivity, but it's not immediately clear how to account for both, and given the uncertainty of the estimates the reliability of the decisions would be doubtful anyway. So we just order by security_level then estimated per-tuple cost, being careful not to change the order when (as is often the case) the estimates are identical.

AND HERE, THIS IS THE ANSWER

```
QualItem *olditem = &items[j - 1];

if (newitem.security_level > olditem->security_level ||
    (newitem.security_level == olditem->security_level &&
     newitem.cost >= olditem->cost))
    break;
items[j] = *olditem;
```

I DON'T GET IT...

